

THE UNIVERSITY OF GHANA



UNIVERSITY OF GHANA

NAME: ASARE ADOM KWAKU

INDEX NUMBER: 22304024

TEAM NAME: SThrive

PROJECT TITLE: SThrive Network

PROGRAMME: COMPUTER SCIENCE

COURSE: SOFTWARE ENGINEERING (DCIT 208)

LECTURER: PROFESSOR MARK ATTA-MENSAH

CLIENT / PROJECT DOMAIN: FAITH-LED WOMEN'S

**PROFESSIONAL NETWORKING AND MEMBERSHIP PLATFORM
(UK-REGISTERED CHARITY)**

Individual GitHub Classroom URL: [https://github.com/Dept-of-Comp-Sci-](https://github.com/Dept-of-Comp-Sci-University-of-Ghana/ia3-use-case-to-story-traceability-card-Adom-Asare)

University-of-Ghana/ia3-use-case-to-story-traceability-card-Adom-Asare

INDIVIDUAL GITHUB PORTFOLIO: <https://github.com/Adom-Asare>

GROUP GITHUB PORTFOLIO: [https://github.com/Dept-of-Comp-Sci-](https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-sthrive/tree/main)

[University-of-Ghana/dcit208-client-project-2026-team-engineering-](https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-sthrive/tree/main)

[repository-sthrive/tree/main](https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-sthrive/tree/main)

DATE: 11-07-26

IA3 — Use-Case-to-Story Traceability Card

UC12 — RSVP to Free Event | SThrive Network

PART A – SELECTED USE CASE

Use Case Name: UC12 — RSVP to Free Event

Use Case ID: UC12

Source evidence:

- D2 SRS (submitted 3 July 2026): Section 2, UC12 — RSVP to Free Event; Section 3, US14 (user story baseline); Section 4, NFR-P02 (atomic capacity check requirement)
- D3 Sprint Backlog: Story US14-A (RSVP atomic capacity check, 5 points, Must, Sprint 3)
- Client meeting transcript, 26 June 2026, Dr. Kinsella — confirmed events are admin-created and open to all members; RSVP and attendance tracking identified as a core requirement replacing the current WhatsApp-based manual process
- Team GitHub issue #35: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-sthrive/issues/35>

Why this use case matters to the client:

She Thrives Network currently announces events across WhatsApp, Instagram, and word of mouth with no formal way to track who is attending or manage capacity. Members express interest informally, and leadership has no visibility into actual attendance numbers. A reliable RSVP system directly replaces this manual, unstructured process and gives Dr. Kinsella's team real data on which events attract engagement. The atomic capacity check is critical because event announcements go out to the full WhatsApp community simultaneously — multiple members attempting to RSVP at the same moment for a limited-capacity session is a realistic, not hypothetical, load scenario for this platform.

PART B – USE CASE CARD

Actor: Free Member / Pro/Elite Member (primary) | Supabase/PostgreSQL, Email Service (secondary)

Goal: Reserve a spot at a free event and receive confirmation, or be placed on a waitlist if the event is at capacity

Trigger: Authenticated member clicks the RSVP button on an event detail page

Preconditions: Member is authenticated with a valid JWT. Event exists in the database with status: scheduled. Member has not already RSVPd to this event. Event ticket price is null or zero (free event).

Main Success Flow

1. Member opens the event calendar, selects an event, and views the event detail page.
2. Member clicks the RSVP button.
3. The Next.js frontend sends POST /api/events/[id]/rsvp with the member's JWT in the Authorization header.
4. Auth middleware verifies the JWT; rate-limiting middleware (Upstash Redis) checks request frequency.
5. The API route executes an atomic SQL UPDATE: increment confirmed_count only if confirmed_count < capacity. Because the check and increment happen in a single indivisible database statement, no two concurrent requests can both pass the capacity check simultaneously.
6. The UPDATE returns the updated row (a slot was available). The API inserts an RSVP record with status: confirmed, returns 201 with { success: true, status: 'confirmed', rsvpId }, and queues a confirmation email.
7. The frontend displays 'You are registered for this event!' and the RSVP button is replaced with a confirmed status label.

Alternate Flow

1. The atomic UPDATE returns nothing (confirmed_count has already reached capacity). The API inserts an RSVP record with status: waitlisted and returns 200 with { success: true, status: 'waitlisted', rsvpId }.
2. The frontend displays 'You have been added to the waitlist.' If a confirmed attendee later cancels, the system automatically promotes the first waitlisted member (ordered by createdAt) to confirmed and sends them a notification email.

Exception / Failure Flow

1. Member has already RSVPd: the compound unique index on (event_id, user_id) catches this at the database level; API returns 409 'You have already registered for this event.' No duplicate RSVP record is created.
2. Supabase connection drops mid-transaction: PostgreSQL rolls back the transaction automatically — confirmed_count is never partially incremented. The API returns 503 'Service temporarily unavailable, please try again' rather than hanging. The RSVP button returns to its default state for retry.

Postconditions

- An RSVP record exists in the rsmps table with status: confirmed or status: waitlisted.

- The event's confirmed_count has been incremented by exactly one (confirmed path only).
- A confirmation or waitlist email has been sent or queued for the member.

PART C – USER STORY

Story ID: US14-A

As a member, I want to RSVP to a free event with a reliable capacity check so that my spot is properly reserved even under high demand.

Source: GitHub issue #35 — <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-sthrieve/issues/35>

Precondition: Member is authenticated. Event exists with status: scheduled. Member has not previously RSVPd.

Postcondition: RSVP record exists with status: confirmed or waitlisted. confirmed_count incremented by exactly one on the confirmed path only.

PART D – ACCEPTANCE CRITERIA

AC1 — Successful Path (Confirmed RSVP)

Given a member is authenticated and the event has available capacity, when they click RSVP and the request is processed, then the system creates an RSVP record with status: confirmed, increments the event's confirmed_count by exactly one, returns a 201 response, and queues a confirmation email to the member.

AC2 — Error / Exception Path (Concurrent Race Condition)

Given two members submit RSVP requests simultaneously for the last available spot, when both requests reach the database at the same moment, then the atomic database operation ensures exactly one RSVP is created with status: confirmed and exactly one is created with status: waitlisted — the event's confirmed_count equals capacity, not capacity + 1, and no overbooking occurs.

AC3 — Duplicate Prevention

Given a member who has already RSVPd attempts to RSVP again, when the request is made, then the system returns 409 with the message 'You have already registered for this event' and no duplicate record is created.

AC4 — Waitlist Promotion

Given a confirmed attendee cancels their RSVP, when the cancellation is processed, then the first waitlisted member (by createdAt) is automatically promoted to confirmed and receives a notification email.

PART E – NON-FUNCTIONAL REQUIREMENT LINK

NFR ID: NFR-P02

NFR Category: Performance — Correctness Under Concurrency

NFR Statement:

The RSVP capacity check must be implemented as a single atomic database operation. The naive check-then-write pattern — read `confirmed_count`, check against capacity, then increment in separate steps — is explicitly prohibited, as it produces a race condition under concurrent load. The implementation must use Supabase's PostgreSQL atomic UPDATE with a WHERE condition to guarantee that the capacity threshold is enforced at the database level, not only the application level, regardless of how many server instances are running simultaneously.

Why it matters for this use case:

She Thrives Network announces events to the full WhatsApp community simultaneously. When a popular session goes live, multiple members will click RSVP within the same second. Without an atomic implementation, two requests can both read `confirmed_count` = 49 (capacity = 50), both pass the check, and both write — producing 51 confirmed attendees for a 50-person event. This is not a theoretical concern: it is a realistic load pattern for this community and a direct consequence of how the organisation currently communicates. The atomic SQL UPDATE eliminates the gap between check and write at the database level, making overbooking structurally impossible.

PART F – GITHUB ISSUE LINK & TRACEABILITY ROW

Issue Title: [Story] US14-A: RSVP atomic capacity check #35

Issue URL: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-sthrieve/issues/35>

Field	Value
Requirement source	D2 SRS UC12; D3 backlog US14-A; client meeting transcript 26 June 2026 (Dr. Kinsella)
Use case	UC12 — RSVP to Free Event
User story	US14-A — As a member, I want to RSVP to a free event with a reliable capacity check so that my spot is properly reserved even under high demand.
NFR	NFR-P02 — Atomic capacity check; performance / correctness under concurrency
GitHub issue URL	https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-sthrieve/issues/35
Acceptance criteria	AC1: Confirmed RSVP on available capacity. AC2: Race condition — only one confirmed under concurrent last-slot requests.
Test idea	Concurrent RSVP race condition test — two simultaneous requests for last slot → exactly one confirmed, one waitlisted.
Status	Planned — Sprint 1 (July 8–21, 2026)

PART G – TEST IDEA

Test Name: Concurrent RSVP Race Condition — Last Available Slot

Test Type: Concurrent Integration Test

Maps to: AC2, NFR-P02

Input / Context

- A test event exists in the database with capacity: 1 and confirmed_count: 0.
- Two distinct authenticated test users (User A and User B) are prepared.
- Both users send POST /api/events/[id]/rsvp simultaneously using Promise.all() to fire both requests at the same millisecond.

Expected Result

- Exactly one RSVP record has status: confirmed.
- Exactly one RSVP record has status: waitlisted.
- The event's confirmed_count equals 1, not 2.
- No duplicate confirmed RSVPs exist in the rsvps table for this event.
- If the atomic implementation is correct, this test passes 100% of the time regardless of request timing. Intermittent failure would indicate the race condition is still present.

PART H – REQUEST-FLOW THINKING

Device / Interface: Any modern browser (desktop or mobile). SThrive Network is a responsive web application — not a native mobile app. Member is viewing the event detail page in the Next.js frontend.

Route / Screen / Endpoint: Frontend screen: /events/[id]. Backend API route: POST /api/events/[id]/rsvp.

Data Sent: HTTPS POST request. Authorization header: Bearer [JWT access token]. URL path parameter: event ID. No request body required for free events — the member's identity is carried in the JWT, not a body field.

Validation Point: Three layers: (1) Auth middleware verifies the JWT and returns 401 if invalid or expired. (2) Upstash Redis rate-limiting checks request frequency per IP. (3) The atomic SQL UPDATE enforces capacity at the database level — this is the critical validation that prevents overbooking regardless of application-level logic.

Database / External Service Touched: Supabase PostgreSQL — events table (atomic UPDATE on confirmed_count); rsvps table (INSERT with confirmed or waitlisted status). Unique compound index on (event_id, user_id) prevents duplicate RSVPs. Email service (Resend/SendGrid) triggered asynchronously for confirmation.

Response: Confirmed: 201 Created { success: true, status: 'confirmed', rsvpId }. Waitlisted: 200 OK { success: true, status: 'waitlisted', rsvpId }. Duplicate RSVP: 409 Conflict. Database failure: 503 Service Unavailable — server fails fast and clearly, never hangs.

Failure Point: The most likely failure point is the Supabase connection dropping mid-transaction. PostgreSQL's transaction guarantees that a dropped connection rolls back

automatically — `confirmed_count` is never left in a partially incremented state. The API catches the connection error and returns 503 rather than leaving the request hanging until the browser times out.

Security / Privacy Concern: The RSVP endpoint is not publicly accessible — JWT verification runs before any database operation. Unauthenticated or tampered requests are rejected at the middleware layer with 401 before touching the database. The event ID in the URL is validated against the database to prevent RSVPs being submitted for non-existent, cancelled, or completed events. No payment data is handled at this endpoint — ticketed events are a separate flow that redirects to Stripe's hosted checkout, so no card data ever passes through the platform's own API.

PART I – AI-DISCLOSURE

AI Used?: Yes

Tool: Claude (Anthropic)

What it helped with:

- Structuring the traceability card to match the IA3 template format
- Verifying technical accuracy of the atomic SQL operation and the 9-step request-flow explanation
- Cross-checking that all required parts (A–H) were addressed per the assignment brief
- Suggesting additional acceptance criteria beyond the initial AC1 and AC2

What I changed:

- All content was verified against the team's own deliverables: D2 SRS (3 July 2026), D3 Sprint Backlog, and the client meeting transcript of 26 June 2026 with Dr. Kinsella
- The user story was taken verbatim from the actual GitHub issue #35, not from AI-generated text
- The failure point description (Supabase connection rollback behaviour) was grounded in real debugging experience from the ThinkBoard project earlier in the year
- Status codes (201, 200, 409, 503) were verified against the team's architecture document

What I personally verified:

- UC12 traces directly to US14-A in the D2 SRS and to GitHub issue #35 in the team repository
- NFR-P02 is taken verbatim from Section 4 of the team's D2 SRS document
- The atomic SQL UPDATE pattern was discussed and understood in full during team architecture planning sessions
- I can explain the race condition, the atomic fix, the waitlist promotion flow, and the failure behaviour orally

PART J – INTEGRITY DECLARATION

I understand the use case, user story, acceptance criteria, traceability links and request-flow reasoning submitted here, and I can explain them orally if called upon.

Typed Name: Asare Adom Kwaku

Student ID: 22304024

Date: 11 July 2026